

BioSync Planner Project Report

An ESP32 based biometric study planning prototype

Course: ICT305

Student name: A.M.C.M.K.Atapattu

Student ID: AS2023970

Submission date: 5 October 2026

Abstract

BioSync Planner combines an ESP32 sensor node with a web application to suggest study tasks and breaks using experimental physiological indicators. A MAX30102 optical sensor provides pulse measurements, while an MPU6050 accelerometer provides motion information. The intended pipeline sends summaries over Wi-Fi using MQTT to a Node.js backend, which calculates a provisional score and updates a React dashboard. The implemented repository includes firmware, task management, a simulator, MQTT ingestion, SQLite persistence, source selection and recommendations. Firmware compilation on the student's MacBook succeeded. Components were acquired and sensor header pins were soldered. At the time of this report, USB device detection was unsuccessful on the MacBook and a university lab computer; firmware upload and live sensor operation therefore remain unverified. The project is reported as an implemented software and assembled hardware prototype awaiting end-to-end validation.

Project purpose and scope

The project explores how a conventional task planner can incorporate sensor context while retaining deadline urgency. It demonstrates embedded acquisition, communication and application integration. The score is an unvalidated demonstration heuristic; it does not measure cognition or diagnose fatigue. The current hardware is a USB-powered finger-contact prototype rather than a completed wrist-worn device.

1 Background and objectives

Student task planners typically organise work by deadlines, duration and priority. BioSync adds physiological context to investigate whether an application can suggest changes in task difficulty and study breaks. Optical pulse measurements require stable contact and can be affected by motion; signal availability is therefore treated separately from the planning score.

Objectives

- Acquire pulse and movement data using an ESP32 and I²C sensors.
- Transmit structured telemetry over MQTT and display it in a web dashboard.
- Rank pending tasks using difficulty suitability and bounded deadline urgency.
- Provide explicitly labelled simulated data for development and demonstrations.
- Handle missing contact, poor signal and stale telemetry without reporting a valid score.
- Evaluate compilation, sensor detection, transport and application behaviour as separate milestones.

2 System architecture

The edge node reads both sensors through a shared I²C bus. Firmware detects optical pulse peaks, estimates pulse intervals and computes experimental pulse RMSSD. The accelerometer contributes a motion estimate. A summary is published approximately every five seconds to biosync/{deviceId}/vitals. Mosquitto runs as the MQTT broker. The backend validates and processes telemetry, stores history in SQLite, applies scoring and scheduling logic, and pushes updates to the frontend with Socket.io. The frontend provides task management, source selection, signal status, recommendations, session controls and recent telemetry history. Optional Claude integration explains recommendations; deterministic advice remains available without an API key.

Layer	Implementation	Responsibility
Edge	ESP32 and Arduino C++	Sensor acquisition and MQTT publishing
Transport	Wi-Fi and Mosquitto	Local telemetry delivery
Backend	Node.js, Express, SQLite	Scoring, ranking, persistence and alerts
Client	React, Vite, TypeScript	Dashboard and task controls

Simulation and hardware use a common sensor contract, but only the selected source updates the live dashboard. The laptop supplies USB power. In the current design, telemetry travels over Wi-Fi rather than over the USB cable.

3 Hardware and wiring

The purchased components are a classic ESP32 development board, MAX30102 MH-ET LIVE module, MPU6050 module, breadboard, jumper wires and a USB-C adapter. The ESP32 is placed beside the breadboard because its width restricts access to adjacent holes. Female jumper ends connect to its headers, and breadboard connections distribute shared signals.

ESP32 pin	MPU6050 pin	MAX30102 pin
3V3	VCC	VIN
GND	GND	GND beside SCL
D21	SDA	SDA
D22	SCL	SCL

This wiring assumes the MAX30102 breakout accepts 3.3 V at VIN and its I²C pull-ups are configured for the ESP32. Its 3V3/1V8 selector was not conclusively verified from the supplied photographs. Confirm the module setting before applying power. Do not use the ESP32 VIN pin as a substitute for the 3V3 connection. Leave MPU6050 XDA, XCL, AD0 and INT disconnected. Leave MAX30102 RD, IRD, INT and its separate 3V3 pin disconnected. Use the GND beside SCL as the main ground connection. Disconnect USB before changing connections. Expected I²C addresses are 0x68 for the MPU6050 with AD0 low and 0x57 for MAX30102.

Assembly observations

Photographs show soldered sensor header pins and four MPU6050 connections matching 3V3, GND, D21 and D22. Photo inspection does not establish electrical continuity or absence of shorts. Final joint inspection, MAX30102 selector confirmation and sensor detection are outstanding.

Component costs

Item	Quantity	Listed cost LKR
ESP32 MD0245	1	1,630
MAX30102 MD0617	1	750
MPU6050 MD0122	1	730
830 point breadboard	1	250
Male to male jumper pack	1	200
Male to female jumper pack	1	190
USB-C adapter	1	80
Total excluding cable and labour	—	3,830

Costs reproduce the order/list screenshot provided by the student. USB data cable and any soldering charges are excluded; this is not a complete verified expenditure statement.

4 Firmware and application logic

Sensor acquisition

MAX30102 setup uses the SparkFun numeric API setup(0x1F, 1, 2, 100, 411, 4096): no FIFO averaging, red and infrared channels, and a configured 100 samples per second. FIFO records are processed to estimate pulse peaks. Pulse intervals are timestamped and traversed chronologically within a rolling 60-second window, using a 240-record capacity. Pulse RMSSD is the square root of the mean squared differences between successive accepted pulse intervals. It is an optical pulse variability estimate, not a validated ECG-derived HRV measurement. The implementation rejects intervals outside its configured range, gates motion artifacts, clears history after contact loss and withholds metrics when insufficient intervals are available. These behaviours still require physical validation. Motion processing compares acceleration magnitude against gravitational acceleration and smooths the resulting activity value into a 0–1 scale. This simplified estimate can miss some changes in orientation and is used as a quality gate rather than as proof of attention.

Telemetry contract

Payload fields are deviceId, sequence, uptimeMs, bootId, source, hrBpm, pulseRmssdMs, motion and quality. Missing metrics are represented as null. The backend adds receivedAt, computes the score centrally and ignores device-supplied scores. The unique boot identifier separates sequence counters across device restarts.

Experimental scoring

With valid, fresh readings, the provisional calculation is:

$$\text{hrvNorm} = \text{clamp}((\text{pulseRmssdMs} - 8) / 52, 0, 1)$$

$$\text{hrNorm} = \text{clamp}((110 - \text{hrBpm}) / 55, 0, 1)$$

$$\text{base} = 100 \times (0.6 \times \text{hrvNorm} + 0.4 \times \text{hrNorm})$$

$$\text{penalty} = \min(20, \max(0, \text{sessionMinutes} - 90) \times 0.3)$$

$$\text{score} = \text{round}(\text{clamp}(\text{base} - \text{penalty}, 0, 100))$$

These constants are demonstration assumptions. Poor quality, no contact, warm-up, missing pulse inputs, excessive motion or stale telemetry results in an unavailable score. HIGH is at least 70, MEDIUM is 50–69, LOW is 35–49 and BREAK_SUGGESTED is below 35.

Task planning and alerts

The scheduler combines difficulty match and deadline urgency, retains pending tasks in break states and provides an explanation for each rank. When sensor context is unavailable, deadline urgency determines ranking. A low-score alert requires ten consecutive minutes of valid low readings during active study; telemetry gaps over 15 seconds interrupt that timer. A separate reminder uses accumulated active study time. These thresholds are configurable development targets rather than validated health recommendations.

5 Implementation and evaluation evidence

The proposal originally described a broader MongoDB-based wearable system. The implemented MVP uses SQLite and USB power to reduce dependencies. BLE, OLED output, battery operation, a wearable enclosure, advanced analytics and clinical validation are deferred. The latest code reviewed in this session was commit 3c258d9; no claim is made about later repository revisions.

Check	Observed result	Interpretation
Mac firmware compilation	938,568 bytes program storage (71%); 50,696 bytes globals (15%)	Compilation succeeded for ESP32 Dev Module; upload not established
Hardware preparation	Components acquired; headers soldered; MPU6050 wiring photographed	Assembly progress documented; sensor operation unverified
Mac USB enumeration	No ESP32 USB interface listed	USB communication not established
Lab computer detection	Student reported no Device Manager change	Board, cable or connection fault remains possible
Lab ESP32 package install	Failed with insufficient disk space	Environment setup incomplete on that computer
Targeted software checks	Gap/pause alerts corrected; hardware acceptance, duplicate rejection, reboot and duration penalty checked	Limited checks only; not full system validation
Repository test suite	README states 28 tests pass	Claim not independently reproduced in this review

Outstanding acceptance tests

Record each result only after performing the test: successful upload; MPU6050 detection at 0x68; MAX30102 detection at 0x57; stable finger pulse readings; successful MQTT publish and backend reception; selected-source dashboard updates; no-contact handling; score invalidation after disconnection; device restart recovery; task persistence; and complete build/test execution.

Current fault investigation

The ESP32 power indicator illuminated, but USB enumeration was not observed on the MacBook or reported on the lab PC. Power presence does not verify the USB data path. A failed cable, adapter, connector or USB-to-UART interface remains possible. The available evidence does not conclusively identify a faulty board. No successful sensor readings, MQTT hardware sessions or end-to-end latency measurement have been recorded.

6 Limitations and future development

The main implementation limitation is the absence of a verified hardware data path. Other limitations include motion-sensitive optical measurement, an unvalidated score, simplified motion processing and fixed pulse detection thresholds. Network reconnect calls remain synchronous in the reviewed firmware and can interrupt acquisition. FIFO batches use processing-time timestamps, so delayed sampling can distort measured pulse intervals. Hardware trials are needed before interpreting the quality or accuracy of measurements. Future development should first resolve USB detection and establish reproducible sensor and transport tests. The next priorities are acquisition independent of network stalls, sample-time-aware pulse processing, stronger signal quality assessment and reliable source-switch integration tests. A wrist-worn version would require a smaller board, stable optical contact, a secure enclosure and correctly regulated protected battery power. Clinical or cognitive claims would require separate research and validation.

7 Conclusion

BioSync Planner implements a modular approach to linking an embedded sensor node with a study planning application. Firmware compilation and hardware preparation are supported by the available evidence. Software modules for simulation, scoring, task ranking, alerts and telemetry handling are present in the reviewed repository. Physical deployment is incomplete because USB communication and sensor operation have not yet been established. The prototype should therefore be presented with simulated input clearly labelled and with hardware integration identified as pending.

8 References and evidence sources

BioSync source repository. <https://github.com/ChethaAtapattu/BioSync>. Reviewed revisions through 3c258d9 during 5 October 2026.

BioSync Project Proposal ICT305 and BioSync Documentation Package. User-supplied planning documents reviewed earlier in this session.

Analog Devices. MAX30102 product information and datasheet.

<https://www.analog.com/en/products/max30102.html>

Espressif Systems. Arduino ESP32 installation documentation.

<https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html>

SparkFun Electronics. MAX3010x Sensor Library source.

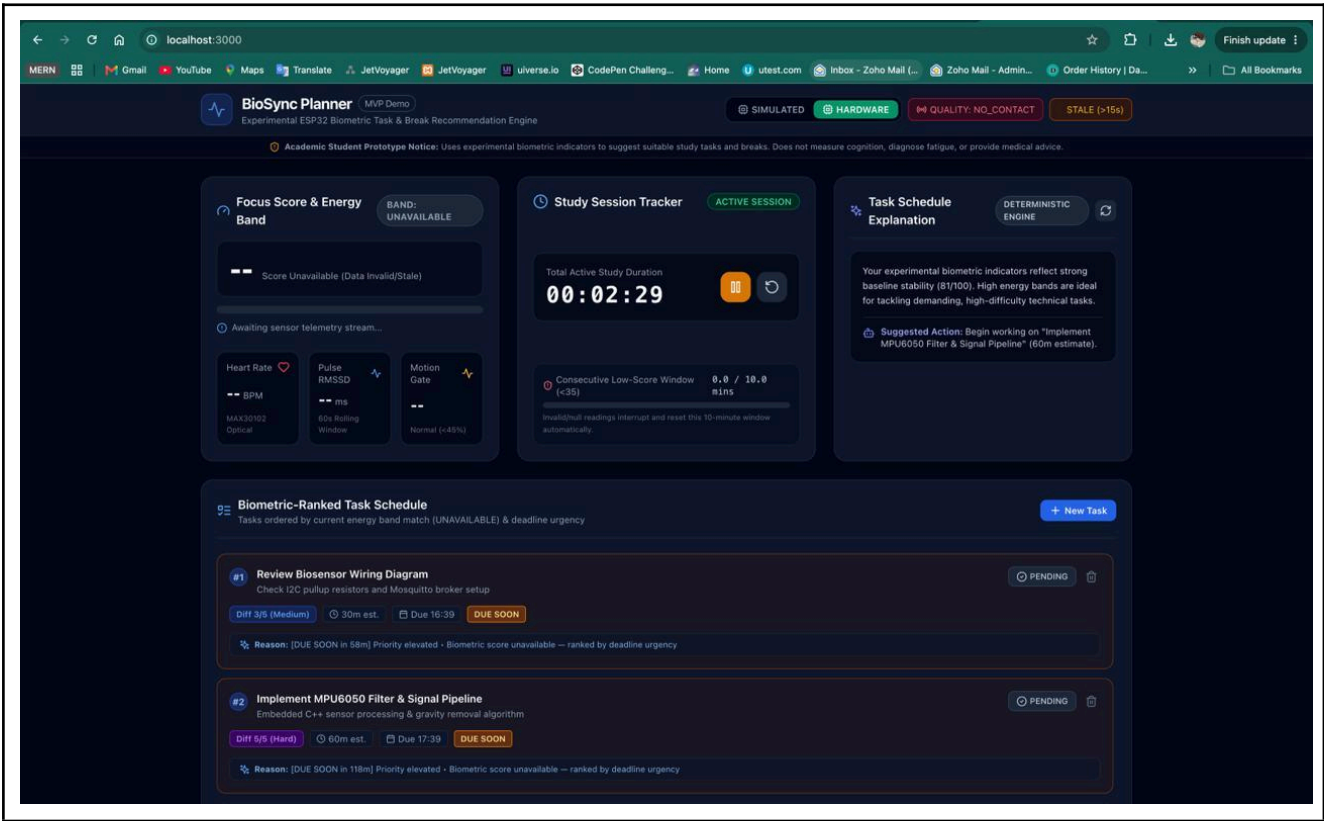
https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library

TRONIC.LK component listings and student order screenshot. ESP32 MD0245, MAX30102 MD0617, MPU6050 MD0122 and accessories. <https://tronic.lk/>

9 Website screenshots

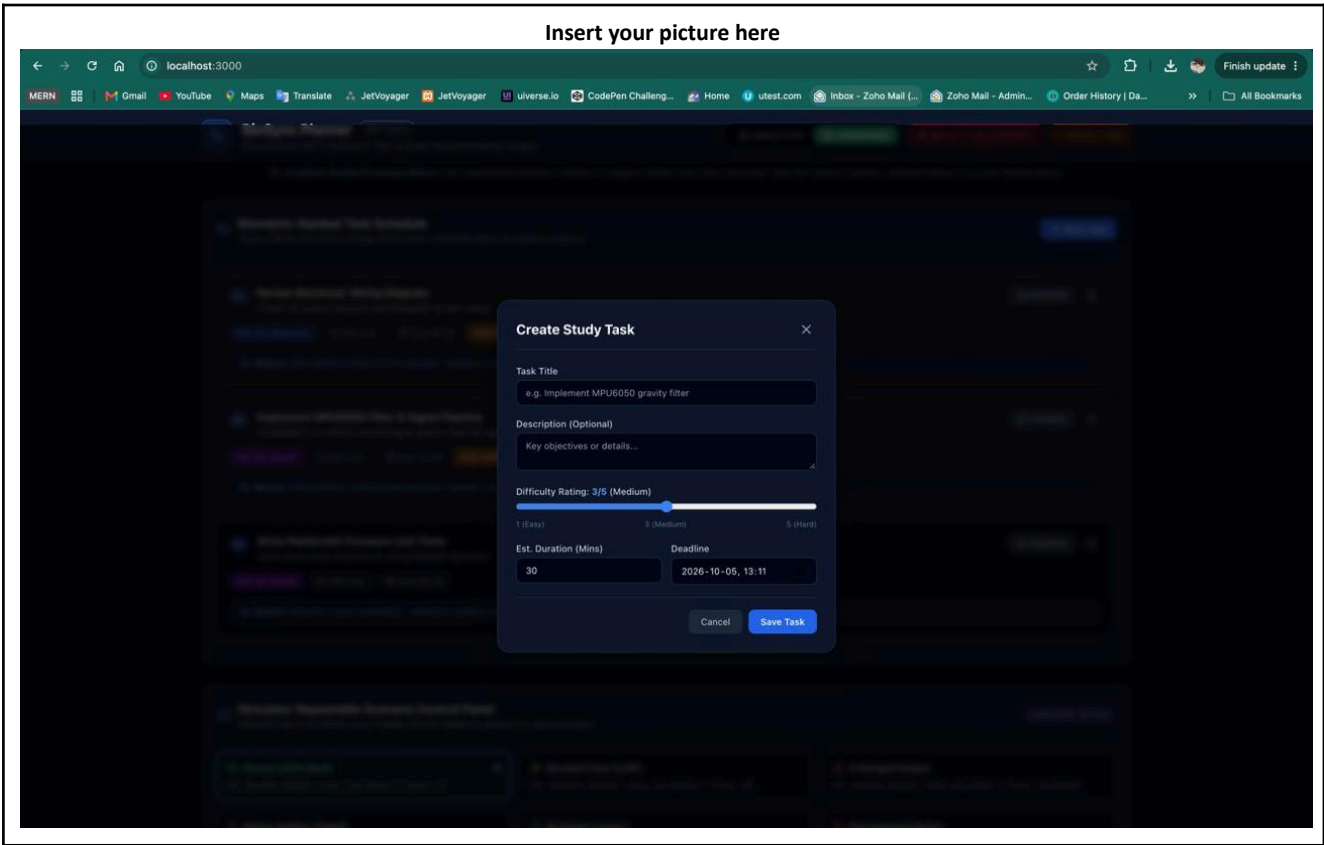
Add screenshots of the BioSync Planner web application. State whether the displayed data is simulated or from hardware.

Figure 1 Web dashboard



Dashboard overview showing source selection, signal status and recommendations.

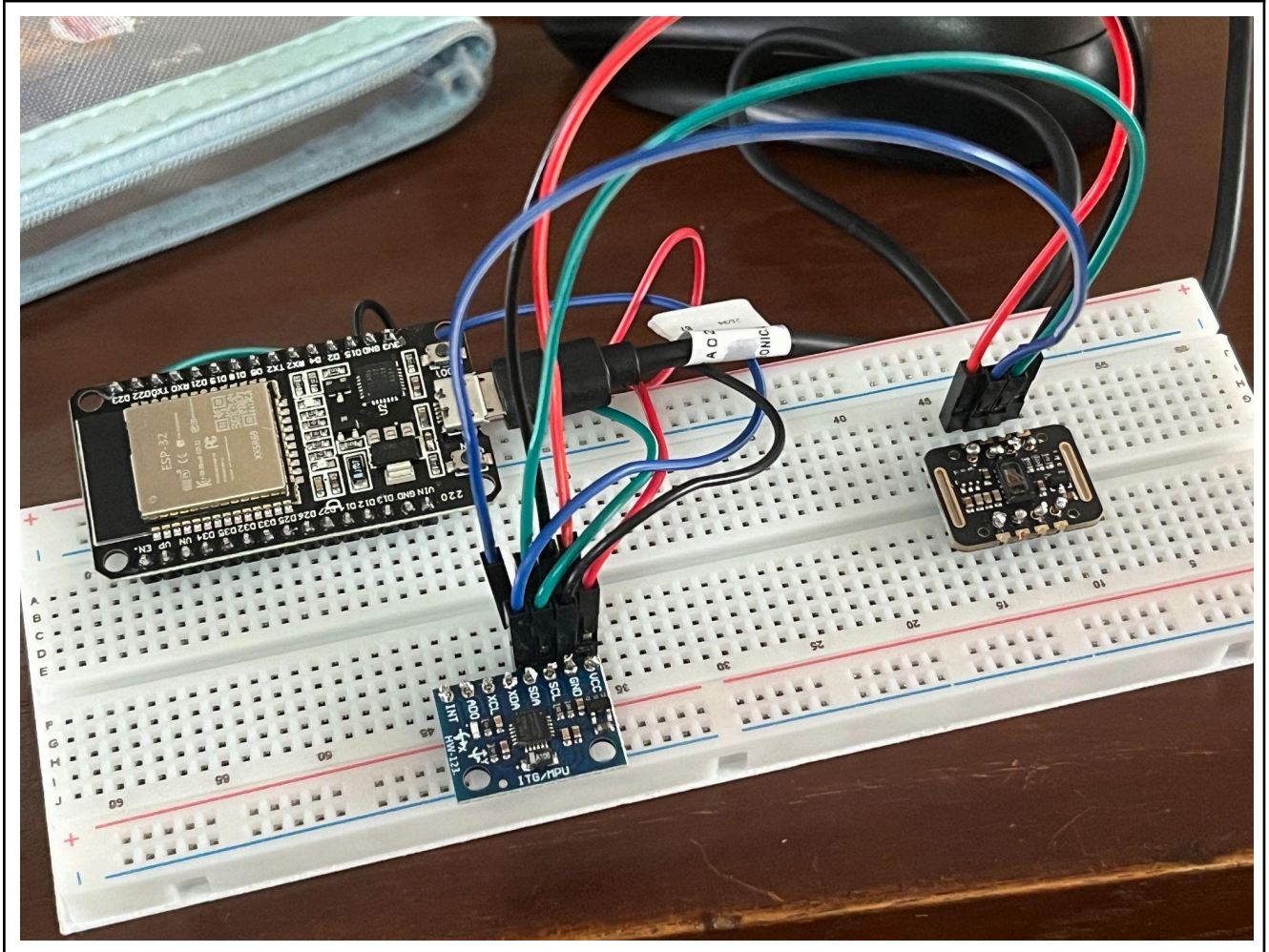
Figure 2 Task planning interface



10 Hardware build photographs

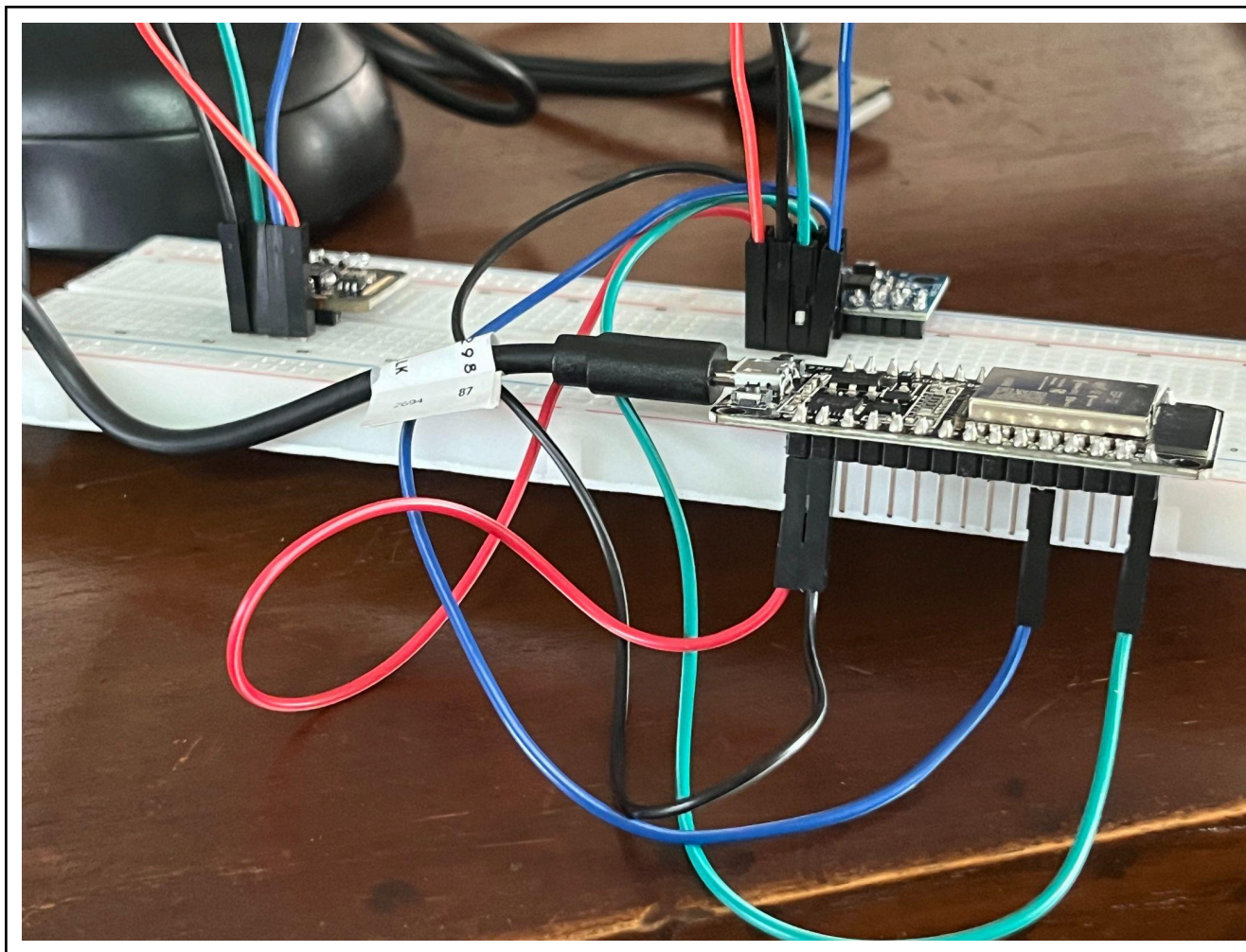
Add photographs documenting the prototype assembly. Describe what each photograph shows and the test status at the time it was taken.

Figure 3 Prototype assembly



Overall ESP32, breadboard, sensors and jumper wire arrangement.

Figure 4 Sensor wiring and soldering



Close-up of sensor connections and soldered header pins.